



CONFIDENTIAL — PRODUCT

QynReact Guide (Incident Intelligence)

Qyenx vOPS HUB — v1.0.0 RC1 · Document v1.0

Document Version	1.0
Software Version	v1.0.0 RC1
Applies To	Qyenx vOPS HUB v1.0.0 RC1
Classification	Confidential — Product
Owner	QynReact Engineering
Status	Released
Last Updated	2026-06-30
Document Type	Module guide

REVISION HISTORY

Version	Date	Notes
1.0	2026-06-30	Initial QynReact guide (Sprint 23): unified Incident Workspace with cross-module intelligence and the fourth production AI adapter.

Table of Contents

- | 1. What QynReact is
- | 2. Cross-module intelligence (no branching)
- | 3. The incident workspace
- | 4. Incident Intelligence (AI surface)
- | 5. API (workspace-scoped, UUID-only)
- | 6. Principles

26 — QynReact Guide (Incident Intelligence)

Audience: Incident commanders, on-call engineers, SREs. **Scope:** QynReact — the unified **Incident Workspace** that reuses Operations Intelligence, Asset Intelligence, and Automation. It is the **fourth production AI adapter**.

1. What QynReact is

QynReact assembles a single incident view from the modules you already run — no data is re-collected and no module-specific branching exists. The incident workspace contains: **Timeline, Assets, Monitoring/Alerts, Recommendations, Automation, Knowledge, Evidence, Resolution, Postmortem**.

UI: `/app/workspaces/:id/qynreact/incidents` .

2. Cross-module intelligence (no branching)

The flow **Alert** → **Asset** → **Incident** → **Automation** → **Knowledge** → **Resolution** is realized by the `CrossModuleOrchestrator` , which iterates the **AI Adapter Registry** and asks each entitled module to build its deterministic context. QynReact itself is excluded from the gather (recursion guard). A future module contributes to incidents automatically once it registers an adapter — there is no `if (module == ...)` anywhere.

3. The incident workspace

Section	Source
Timeline	Incident timeline entries (notes, status changes, linked automation, evidence).
Cross-module	<code>qynsight</code> , <code>qynasset</code> , <code>qynrun</code> (and future) contexts via the registry.
Automation	Executions linked to the incident (from the Automation Platform).
Knowledge	Honest "not collected" until QynKnow is connected — never fabricated.
Resolution / Postmortem	Operator-authored; postmortem can be AI-drafted (editable).

4. Incident Intelligence (AI surface)

- **Copilot** — grounded Q&A over the assembled incident evidence (reuses Quenyx AI conversations).
- **Recommend** — evidence-based next response actions, citing cross-module evidence and the auditable automation-learning success rates; destructive actions flagged for approval.
- **Postmortem** — an editable draft (summary, impact, timeline, root-cause **hypothesis**, action items) generated only from the incident evidence.

Nothing is auto-executed. Recommended actions run through QynRun's approval gate.

5. API (workspace-scoped, UUID-only)

Base: `/api/qynreact` . RBAC via `accessAi` ; AI surfaces require the `can_use_ai` capability.

Method	Endpoint	Purpose
GET/POST	<code>/incidents</code>	List / open incidents.
GET	<code>/incidents/{uuid}</code>	Unified incident workspace.
PUT	<code>/incidents/{uuid}</code>	Update status / resolution / postmortem.
POST	<code>/incidents/{uuid}/timeline</code>	Add a timeline entry.
POST	<code>/incidents/{uuid}/copilot</code>	Incident Copilot.
POST	<code>/incidents/{uuid}/recommend</code>	Response recommendations.
POST	<code>/incidents/{uuid}/postmortem</code>	Postmortem draft.

6. Principles

Reuse over duplication · evidence or honesty · no auto-execution · workspace isolation · RBAC · UUID-only · audited · EN/AR. See the Incident Response Guide (27) for the operational playbook.